POLAR-SGD: Efficient Cross-Datacenter Training via Prefix-based Communication Overlap and Error Correction

Anonymous Authors[1]

Abstract

Training large language models (LLMs) requires a large number of GPUs and massive amounts of data. Cross-datacenter (Cross-DC) training not only alleviates GPU resource pressure in a single DC but also avoids transferring large volumes of data across regions. In Cross-DC training scenarios, using data parallelism (DP) as the primary inter-DC parallelization strategy can better cope with the limited network bandwidth between DCs than approaches centered on other parallelization strategies such as pipeline parallelism (PP). However, the blocking All-Reduce operations for gradient synchronization among DP groups, still becomes a severe performance bottleneck, underutlizing GPUs.

To address this, we propose POLAR-SGD, a novel training method that eliminates the synchronization bottleneck by overlapping communication with computation. The core idea of POLAR-SGD is to treat the backward pass of each microbatch within a mini-batch as a prefix gradient accumulation phase and immediately initiate a non-blocking All-Reduce for its gradients. This allows gradient communication to proceed in parallel with the computation of the next micro-batch. To resolve the gradient divergence introduced by this asynchrony, POLAR-SGD incorporates a lightweight predictive error-correction mechanism that compensates for gradient staleness, thereby maintaining training stability and achieving convergence performance highly matched with synchronous SGD.

We evaluated POLAR-SGD on models including LLaMA-2 7B on a cluster of 32 A100 GPUs, emulating real Cross-DC conditions (30 Gb/s bandwidth, 50ms latency) via network throttling. Experiments show that POLAR-SGD achieves a **1.87×** improvement in training throughput compared to the baseline without techniques of POLAR-SGD. POLAR-SGD comes with nearly no loss in model quality and its convergence curve is nearly identical to the traditional synchronous baseline. Our code base see A.1

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Large language models (LLMs) have become foundational to a wide range of artificial intelligence applications (Bommasani et al., 2021; Brown et al., 2020; Touvron et al., 2023). To achieve superior performance, practitioners are training increasingly larger models on ever-expanding datasets, driving an urgent need for scalable and efficient distributed training paradigms (Narayanan et al., 2021).

This scaling trend is dramatic: as shown in Table 1, model parameters have grown from 0.34B (BERT-large, 2018) to 671B (DeepSeek-R1, 2024), while training datasets have expanded from 3.3B to 2 trillion tokens (Devlin et al., 2019; Brown et al., 2020; Chowdhery et al., 2022; Touvron et al., 2023; DeepSeek-AI, 2025). Training such models requires thousands of accelerators (GPUs/TPUs) operating continuously for weeks or months (Shoeybi et al., 2019; Narayanan et al., 2021). However, physical constraints within a single data center—including power density, cooling capacity, and physical space—make **cross-datacenter (Cross-DC) training** across geographically distributed facilities an emerging imperative (Liang et al., 2024).

Table 1. Evolution of language model scale (parameters and training data).

Model	Release	Parameters	Dataset
BERT-large	2018	0.34B	3.3B tokens
GPT-2	2019	1.5B	10B tokens
GPT-3	2020	175B	300B tokens
PaLM	2022	540B	780B tokens
LLaMA	2023	65B	1.4T tokens
DeepSeek-R1	2024	671B	2T tokens

To address this computational demand, the de facto stan-

For ICML 2026 reviewers: When reviewing this paper, please follow the policy for the use of LLMs as specified on your Reviewer Console. Note that the assigned LLM policy might differ from the policy you indicated during reviewer registration.

dard for training large models employs a hybrid of Data Parallelism (DP) and Pipeline Parallelism (PP) (Narayanan et al., 2021; Shoeybi et al., 2019), often augmented with intra-stage strategies such as Tensor Parallelism (TP), Sequence/Context Parallelism (SP/CP) (Narayanan et al., 2021), and Expert Parallelism (EP) (Lepikhin et al., 2021). In this approach, a large global batch is divided into multiple micro-batches that are fed sequentially through pipeline stages to maximize hardware utilization (Narayanan et al., 2021; Shoeybi et al., 2019). In cross-DC scenarios, this paradigm can be further extended: employing data parallelism across different domains and pipeline parallelism within each domain, coupled with additional intra-stage parallel strategies, can theoretically achieve even better throughput (see 2.2.1 and A.2 for details).

However, in the Cross-DC setting, this widely adopted strategy encounters a critical challenge: the network characteristics of Wide Area Networks (WAN) differ drastically from intra-datacenter . Within a data center, high-speed interconnects such as InfiniBand or RDMA-capable Ethernet provide 200-800 Gb/s bandwidth with sub-millisecond latency. In contrast, cross-datacenter links rely on WAN infrastructure, typically offering only 10-30 Gb/s bandwidth with 30-100 ms latency, a $10 - 40\times$ bandwidth reduction and $30 - 100\times$ latency increase (Liang et al., 2024). This network disparity renders the blocking All-Reduce synchronization in standard DP+PP a severe bottleneck.

In standard implementations (e.g., the 1F1B schedule (Narayanan et al., 2021)), gradient synchronization across data-parallel replicas, typically a blocking All-Reduce, commences only after all micro-batches have completed their forward and backward passes. Within a single data center, high-bandwidth, low-latency interconnects often enable this communication to overlap effectively with computation. In contrast, under Cross-DC conditions, constrained bandwidth and elevated latency substantially prolong the All-Reduce operation, creating pronounced end-of-step idle periods during which accelerators stall while awaiting inter-datacenter communication. This effect fundamentally limits scalability and degrades training throughput.

Contributions. We propose **Polar-SGD**, a prefix-based early synchronization framework designed to address the communication bottleneck in Cross-DC data-parallel and pipeline-parallel (DP+PP) training. Our contributions are:

1. **Pipelined Gradient Communication:** Rather than waiting until the end of each mini-batch to synchronize, Polar-SGD triggers non-blocking All-Reduce operations on prefix-accumulated gradients during micro-batch computation, fully overlapping communication with computation. This pipelined strategy effectively eliminates accelerator idle time and alleviates WAN-induced synchronization delays (Liang et al., 2024).

2. **Predictive Error Correction:** To address gradient staleness resulting from the relaxed synchronization schedule (Stich et al., 2018; Modoranu et al., 2024; Bao et al., 2025; Xu & Huang, 2022; Xu et al., 2021; Karimireddy et al., 2019), we introduce a lightweight error-feedback mechanism that tracks the discrepancy between locally predicted and globally synchronized gradients, applying corrective terms in subsequent updates. This stabilizes training and ensures robust convergence, as corroborated by theoretical analysis and ablation studies.

3. **Comprehensive validation:** We provide theoretical convergence analysis and empirical evaluation. In simulated Cross-DC scenarios (30 Gb/s bandwidth, 50 ms latency), Polar-SGD achieves a $1.87\times$ throughput improvement over standard DP+PP while matching model quality.

This work charts a practical and theoretically grounded path for efficient, large-scale model training across wide-area networks.

2. Background and Motivation

Training large language models across geographically distributed datacenters (Cross-DC) introduces system-level challenges that differ fundamentally from those in single-datacenter environments. In particular, Wide Area Networks (WANs) exhibit orders-of-magnitude lower bandwidth, higher latency, and greater variability than intra-datacenter interconnects, fundamentally altering the performance characteristics of distributed training. This section first reviews the design space of Cross-DC training systems and then motivates the design choices underlying this work.

2.1. Background

2.1.1. Cross-Datacenter Training Characteristics

In Cross-DC settings, compute clusters are interconnected via WAN links that typically provide only 10–30 Gb/s bandwidth with round-trip latencies of 30–100 ms, in stark contrast to intra-datacenter fabrics such as InfiniBand or RDMA-capable Ethernet, which routinely deliver 200–800 Gb/s bandwidth with sub-millisecond latency. These network characteristics fundamentally change the cost model of distributed training: communication that is tolerable within a datacenter can dominate end-to-end iteration time when performed across datacenters. As a result, training systems must tolerate both persistent communication constraints while often operating on geographically partitioned datasets that cannot be easily centralized due to data locality, regulatory, or operational constraints.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

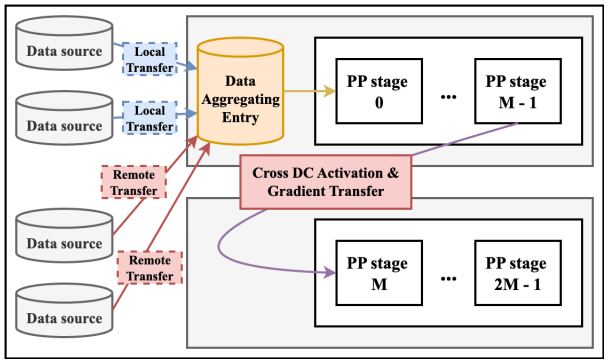

Figure 1. Cross-DC pipeline parallelism. All training data must be aggregated at a single entry-point datacenter, creating severe ingress bottlenecks.

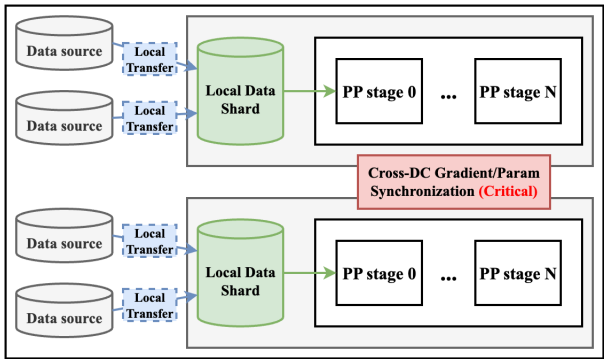

Figure 2. Cross-DC data parallelism with intra-datacenter pipeline parallelism (our setting). Each datacenter processes local data and synchronizes only gradients across the WAN.

2.1.2. Parallelism Backbones Indicate a Fundamental Design Trade-off

To scale training under these constraints, existing Cross-DC systems adopt different parallelism backbones that define how computation and communication are organized across datacenters. Pipeline-parallel–centric designs partition the model into sequential stages and stream micro-batches through the pipeline, enabling training of models that exceed the capacity of individual devices or clusters. Several Cross-DC systems(Chen et al., 2025; Dai et al., 2025; Deng et al., 2025) adopt pipeline parallelism as their primary backbone, leveraging its high compute efficiency under ideal communication conditions.

In contrast, data-parallel-centric designs replicate the model across datacenters and partition the dataset instead. Each datacenter performs forward and backward computation locally, while periodically synchronizing model updates across sites. This approach aligns naturally with data locality and minimizes cross-datacenter data movement during the main computation phase, at the cost of explicit synchronization.

2.1.3. Synchronization and Communication Practices

Independent of the chosen parallelism backbone, Cross-DC training systems differ in how they synchronize model updates across datacenters. Fully synchronous schemes enforce global barriers at every iteration, preserving strict optimization semantics but exposing WAN latency directly on the training critical path. Asynchronous schemes decouple computation from synchronization, allowing workers to proceed independently, but introduce gradient staleness and potential model divergence. Periodic or local synchronization strategies amortize communication cost by performing multiple local updates before synchronization, trading reduced communication frequency for increased optimization drift.

Collectively, prior work reveals a fundamental tension: synchronization is either frequent and expensive under WAN conditions, or relaxed in ways that perturb the optimization trajectory. This tension motivates a closer examination of how parallelism choices and synchronization interact in Cross-DC environments.

2.2. Motivation

2.2.1. Rethinking Parallelism Choices in Cross-DC Training

Pipeline parallelism is widely regarded as compute-efficient when training is evaluated from a computation-only perspective. By overlapping forward and backward execution across stages, PP can achieve high device utilization under stable, low-latency communication. However, in Cross-DC settings, training efficiency is shaped not only by computation but also by data movement and network dynamics.

In PP-centric Cross-DC designs, training samples or activations must enter the pipeline and be propagated across stages that may span multiple datacenters. Pipeline execution enforces strict ordering constraints between stages, causing delays in cross-datacenter communication to directly stall downstream computation. As illustrated in Figure 1, data availability, pipeline progress, and WAN performance become tightly coupled: a slowdown or transient latency spike affecting a single stage can cascade through the pipeline and amplify end-to-end iteration time.

In contrast, data-parallel centric designs localize forward and backward computation within each datacenter. Data loading, preprocessing, and model execution proceed without reliance on WAN communication, and cross-datacenter interaction is confined to synchronization phases outside the main execution path (Figure 2). As a result, DP-centric designs are inherently more robust to network variability.

Our theoretical proofs in the A.2 reveal that in scenarios

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

involving training large amounts of data across data centers, the throughput using cross-data center data parallelism is significantly higher than that of cross-data center pipeline parallelism. These observations motivate a hierarchical DP+PP architecture, in which data parallelism is used across data centers to ensure locality and robustness, while pipeline parallelism is employed within each data center to scale model capacity(Kloudas et al., 2015).

2.2.2. Synchronization Becomes the Dominant Bottleneck

While a DP+PP backbone addresses data locality and execution robustness, it exposes a new performance bottleneck: cross-datacenter synchronization. In DP-based Cross-DC training, each datacenter independently executes forward and backward passes on local data, but must synchronize model updates to maintain a consistent global model state.

Under WAN conditions, synchronization latency is no longer negligible. A breakdown of iteration time under a DP+PP configuration (Figure 3) shows that synchronization in cross-DC scenario can account for a substantial fraction of total iteration time. Consequently, GPUs remain idle for a large fraction of time, waiting for synchronization to complete.

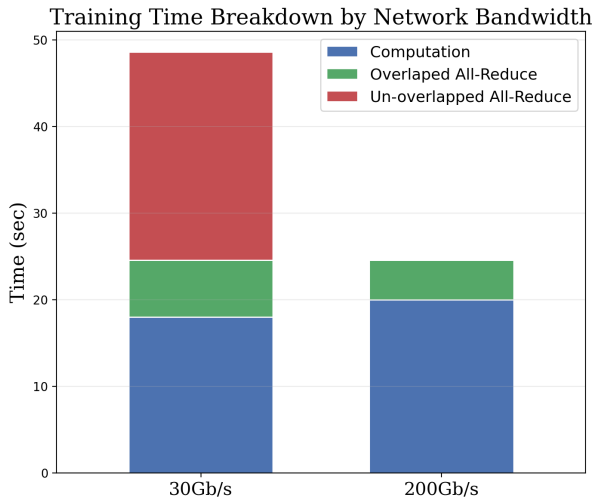

Figure 3. Breakdown of computation and communication overhead per training step under a DP+PP configuration across and within datacenters.

Importantly, existing communication overlap mechanisms in standard data-parallel frameworks(Zhao et al., 2023; Li et al., 2020; Rajbhandari et al., 2020) only overlap gradient communication within a single backward pass. Under WAN-scale latency, the tail of the synchronization inevitably extends beyond the available computation, making end-of-step blocking unavoidable. Thus, while DP+PP mitigates data-movement inefficiencies, it shifts the primary performance bottleneck to synchronization.

2.2.3. Decoupling Synchronization Without Sacrificing Optimization Semantics

A natural response to the synchronization bottleneck is to adopt partially asynchronous methods such as Local-SGD(Stich, 2019). However, these schemes exhibit two principal limitations: unresolved performance bottlenecks and an increased risk of model drift during training.(Karimireddy et al., 2020; Gu et al., 2023)

From a performance perspective, Local-SGD primarily reduces the frequency of cross-datacenter synchronization rather than eliminating the communication costs inherent to distributed optimization. Periodic global aggregation remains unavoidable. The consequence is sustained GPU idle time.

From an optimization/precision perspective, increasing the number of local steps elevates the risk of model drift. Gradients computed on these stale local replicas introduce systematic bias; this bias can slow convergence and degrade practical generalization performance.

These observations suggest that merely weakening synchronization is insufficient. Conversely, some fully asynchronous schemes(Jaghouar et al., 2024; Douillard et al., 2025) eliminate compute bubbles but incur severe parameter staleness that is difficult to correct. An effective cross-datacenter training system therefore needs to decouple synchronization from the training critical path while preserving convergence robustness comparable to fully synchronous optimization, rather than relying on strict equivalence of update sequences.

This insight motivates the design of POLAR-SGD.

3. POLAR-SGD Design

In this section, we present POLAR-SGD (Predictive Overlap in Local SGD with Adaptive Redundancy), a novel approach designed to address the communication bottleneck in DP+PP hybrid parallel training. The core principle is to transform blocking gradient synchronization into a pipelined communication process that overlaps with computation, while maintaining model convergence stability through a predictive error-correction mechanism.

3.1. Notation

We first introduce the key notation used throughout this section. Table 2 summarizes the symbols and variables employed in our algorithms.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

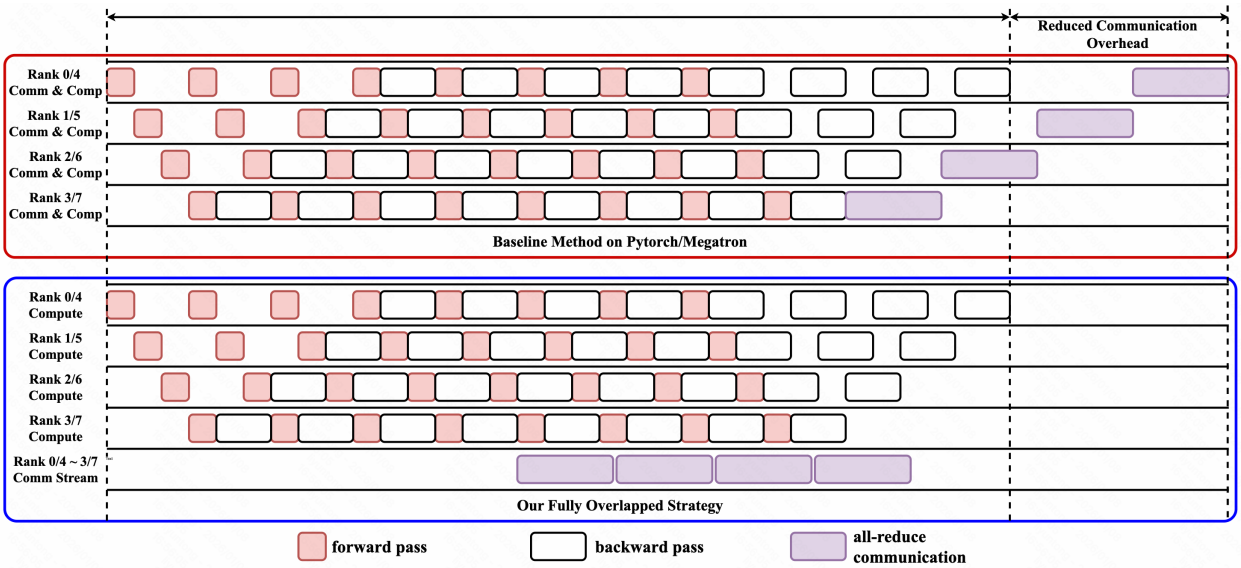

Figure 4. **Execution timeline comparison under DP+PP.** We illustrate one global iteration with M=8 micro-batches, DP size = 2, and PP size = 4. Each row labeled as *Rank* $x/x{+}4$ represents the same pipeline stage replicated across the two data-parallel ranks (e.g., Rank $0/4$ denotes stage 0 on the two DP replicas). **Top: Baseline (PyTorch/Megatron).** Gradient synchronization is performed in a blocking manner near the end of the mini-batch, forming a long All-Reduce tail (purple) after most backward computations (white) have finished, which leaves accelerators under-utilized. **Bottom: POLAR-SGD.** We launch non-blocking All-Reduce after the prefix micro-batches up to cutoff τ are accumulated, and offload the collective to a dedicated communication stream, thereby overlapping All-Reduce (purple) with subsequent backward computations (white) and shrinking the iteration tail.

3.2. Overview of POLAR-SGD

Figure 4 compares the execution timelines of a standard DP+PP training loop and our POLAR-SGD design. The example uses M=8 micro-batches per mini-batch, DP size = 2, and PP size = 4 (i.e., 8 global ranks in total). In the figure, each row labeled as *Rank* $x/x{+}4$ aggregates the behavior of the same pipeline stage across the two data-parallel replicas; for instance, *Rank* $0/4$ corresponds to pipeline stage 0 on the two DP ranks.

Baseline DP+PP suffers from a synchronization tail. As shown in Figure 4(top), pipeline parallelism interleaves forward (red) and backward (white) computations across stages and micro-batches, but data-parallel gradient synchronization is typically deferred until the end of the mini-batch after all M micro-batches have completed backward. Consequently, a large blocking All-Reduce phase (purple) appears near the end of the iteration, leading to significant accelerator idle time and reduced throughput.

POLAR-SGD eliminates the tail via prefix-based early synchronization. Figure 4(bottom) illustrates our key idea: we trigger *exactly one* asynchronous All-Reduce earlier within the iteration, after only a prefix of micro-batches has finished backward. While the collective runs in the background, the main compute stream continues executing backward for the remaining micro-batches. The remaining $(M - \tau)$ micro-batches are incorporated through an error-feedback residual carried to the next iteration. Thus, POLAR-SGD does not discard gradient information; it postpones the utilization of a portion of the mini-batch to enable communication–computation overlap with stable optimization behavior.

Algorithm 1 POLAR-SGD: Prefix-based pipelined gradient communication (one All-Reduce per iteration)

Global variables: accumulated gradient g_a, prefix snapshot $g^{(\tau)}$, communication handle h
Procedure: OnBackwardMicroBatch$(t, m, g_{r,t,m})$
$g_a \leftarrow g_a + g_{r,t,m}$ {PP gradient accumulation}
if $m == \tau$ **then**
  $g^{(\tau)} \leftarrow g_a$ {snapshot prefix gradient}
  (build send buffer $s_{r,t}$ as in Algorithm 2)
  $h \leftarrow$ AsyncAllReduce$(s_{r,t})$
end if
if $m == M$ **then**
  wait(h); {obtain $g_{\text{sync},t}$ (mean across DP workers)}
  OptimizerStep$(g_{\text{sync},t})$
  $g_a \leftarrow 0$
  $h \leftarrow NULL$
end if

3.3. Pipelined Gradient Communication

After the backward pass of each micro-batch m finishes, we

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Table 2. Notation and terminology used in POLAR-SGD

Symbol	Description
M	Number of micro-batches per mini-batch
m	Micro-batch index
t	Global iteration index
r	Worker index (rank) within a DP group
$g_{r,t,m}$	Gradient of worker r at iteration t for micro-batch m
$g_{r,t}^{(\tau)}$	Prefix accumulated gradient on worker r up to micro-batch τ
$g_{r,t}^{(M)}$	Full accumulated gradient on worker r up to micro-batch M
$g_{\text{pred},r,t}$	Predicted (scaled) gradient sent to All-Reduce at iteration t
$g_{\text{sync},t}$	Synchronized gradient from All-Reduce (mean across DP workers)
$e_{r,t}$	Error buffer on worker r at iteration t
$s_{r,t}$	Send buffer for communication at iteration t
h_t	Communication handle at iteration t
w_t	Model weights at global iteration t
η	Learning rate
N	DP group size (number of workers participated in All-Reduce)
T	Number of global iterations
τ	Communication cutoff micro-batch index

accumulate its gradient into a running buffer. When $m = \tau$, we launch a non-blocking All-Reduce on a communication buffer built from the current prefix-accumulated gradient. The `GradientUpdateJudge` function returns true only at the end of the mini-batch ($m = M$), where we wait for the asynchronous communication to complete and then apply the optimizer step.

Choosing the cutoff τ. Let T_{ar} be the estimated All-Reduce latency (mean) for the communicated gradient tensor, and let $T_{\text{comp}}(\tau)$ denote the remaining computation time after launching communication at micro-batch τ within the same iteration. We select the earliest cutoff that can fully hide communication:

$$\tau^* \triangleq \min\{\tau \in \{1, \dots, M\} : \; T_{\text{comp}}(\tau) \geq T_{\text{ar}}\}, \quad (1)$$

In practice, T_{ar} and $T_{\text{comp}}(\tau)$ can be obtained from online profiling with a sliding-window average. Intuitively, smaller τ yields more overlap but increases the reliance on error feedback; larger τ reduces approximation and recovers the baseline behavior.

3.4. Predictive Error Correction

Launching All-Reduce early means that the parameter update is only based on a *prefix* gradient. POLAR-SGD addresses this by forming a full-batch-scale *predicted* gradient at $m = \tau$ and maintaining an error buffer that captures the mismatch between the prediction and the full accumulation.

Prefix and full gradients. On worker r at iteration t, define the prefix accumulated gradient and the full accumulated gradient as

$$g_{r,t}^{(\tau)} \triangleq \sum_{m=1}^{\tau} g_{r,t,m}, \qquad g_{r,t}^{(M)} \triangleq \sum_{m=1}^{M} g_{r,t,m}. \quad (2)$$

At $m = \tau$, g_a equals $g_{r,t}^{(\tau)}$; after $m = M$, g_a equals $g_{r,t}^{(M)}$.

Predicted gradient and error update (full-batch scale). At the cutoff point $m = \tau$, we form the send buffer

$$s_{r,t} = \frac{M}{\tau}\left(g_{r,t}^{(\tau)} + e_{r,t}\right), \quad (3)$$

and record the predicted (pre-reduction) gradient as

$$g_{\text{pred},r,t} \triangleq s_{r,t}. \quad (4)$$

We then perform a non-blocking All-Reduce *mean* over $s_{r,t}$ across the DP group and obtain the synchronized gradient

$$g_{\text{sync},t} = \frac{1}{N}\sum_{r=1}^{N} g_{\text{pred},r,t}. \quad (5)$$

After finishing all M micro-batches, we update the error buffer by the mismatch between the *full* accumulated gradient and the predicted gradient:

$$e_{r,t+1} = g_{r,t}^{(M)} - g_{\text{pred},r,t}. \quad (6)$$

This residual contains the information from the remaining $(M - \tau)$ micro-batches (and any prediction error) and is injected into the next iteration through Eq. (3). Thus, POLAR-SGD does not discard gradients from later micro-batches; it defers their effect via a controlled error-feedback pathway.

3.5. Convergence Analysis

Under standard assumptions (L-smoothness, unbiased gradient, bounded variance), we prove POLAR-SGD achieves the same asymptotic convergence rate as synchronous SGD (see A.4 for full proof). The key theorem shows the average gradient norm bound only includes an additional constant term controlled by the error buffer, confirming stable optimization behavior.

4. Evaluation

In this section, we conduct a comprehensive evaluation of POLAR-SGD under a realistic cross-datacenter (Cross-DC) training setting.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Algorithm 2 POLAR-SGD: Error feedback with prefix scaling

Global variables: error buffer $e_{r,t}$, predicted gradient $g_{\text{pred},r,t}$, cutoff τ, M
Initialize: $e_{r,0} \leftarrow 0$
Procedure: BuildSendBufferAtCutoff$(t, g_{r,t}^{(\tau)})$
$s_{r,t} \leftarrow \frac{M}{\tau}\left(g_{r,t}^{(\tau)} + e_{r,t}\right)$
$g_{\text{pred},r,t} \leftarrow s_{r,t}$
return $s_{r,t}$ {passed to AsyncAllReduce in Algorithm 1}
Procedure: UpdateError $(t, g_{r,t}^{(M)})$
$e_{r,t+1} \leftarrow g_{r,t}^{(M)} - g_{\text{pred},r,t}$

Table 3. Experimental setup.

Component	Specification
Model	LLaMA-2 7B
Dataset	WikiText-103
Hardware	32× NVIDIA A100 (40GB) GPUs across 4 nodes
Network	Bandwidth: 30 Gb/s; RTT latency: 50 ms
Parallelism	Hybrid DP+PP (PP=8, DP=4); batch=256; microbatch=32
Software	PyTorch 2.5; NCCL 2.23; CUDA 12.1
Metrics	Throughput (tokens/s), training loss

4.1. Experimental Setup

Table 3 summarizes our experimental configuration. We compare POLAR-SGD against a standard implementation, denoted as **Baseline DDP (Standard DP+PP)**. This baseline employs a 1F1B pipeline schedule and performs a blocking All-Reduce to synchronize gradients at the end of each mini-batch. Under WAN-like Cross-DC conditions, such barriered synchronization typically induces substantial stalls and reduces effective GPU utilization.

In addition, we include a representative relaxed synchronization baseline, **LSGD** ($k = 4$), to highlight how delayed/local synchronization can alter training dynamics.

4.2. Main Results

We report end-to-end throughput in tokens/s in Table 4. For POLAR-SGD, we additionally report the normalized speedup over the Baseline DDP.

Convergence comparison by steps. Figure 5 reports the smoothed loss curves aligned by training steps over a common range. All methods exhibit a consistent overall trend: the loss decreases steadily and reaches a plateau after roughly 200 steps. Importantly, POLAR-SGD tracks the Baseline DDP closely across most of training, indicating that the proposed approach preserves stable optimization behavior despite introducing more aggressive communication/synchronization strategies.

Table 4. End-to-end throughput under Cross-DC network constraints.

Method	Throughput (tokens/s)	Speedup
DDP+1F1B	5,350	1.00×
LSGD ($k = 4$)	9,433	1.76×
POLAR-SGD	10,016	1.87×

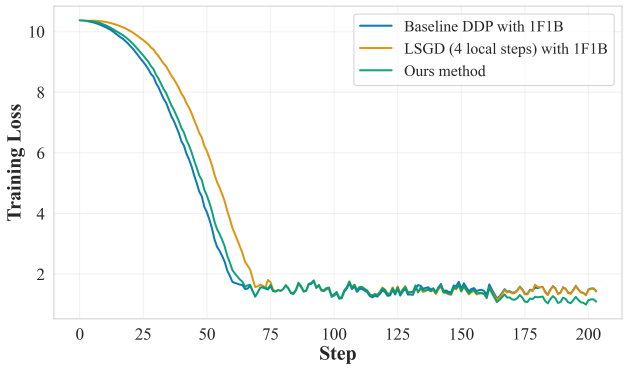

Figure 5. Training loss versus steps on WikiText-103 (smoothed over a common step range). POLAR-SGD closely tracks the Baseline DDP, while LSGD shows a mild mid-to-late stage deviation, indicating that POLAR-SGD better preserves step-wise convergence stability under Cross-DC constraints.

End-to-end convergence comparison by wall-clock time. xs The advantage of POLAR-SGD is more pronounced when comparing convergence as a function of wall-clock time. As shown in Figure 6, POLAR-SGD reduces the loss substantially faster than the Baseline DDP under the same relative time budget.

Taken together, Figures 5 and 6 support two conclusions. First, **convergence stability** is preserved: when aligned by steps, POLAR-SGD closely matches the baseline, showing no signs of instability or divergence. Second, **model quality is not sacrificed for throughput**: POLAR-SGD achieves comparable training loss while substantially reducing wall-clock time.

4.3. Ablation Study

We conduct ablation experiments on the two core mechanisms of this paper. As shown in 7, for training with identical configurations, the three variants: without gradient scaling, without the error feedback strategy, and without both gradient scaling and error feedback exhibit a slight degradation in terms of convergence degree and convergence stability compared to our original method. Specifically, the variant without gradient scaling and error feedback shows a

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

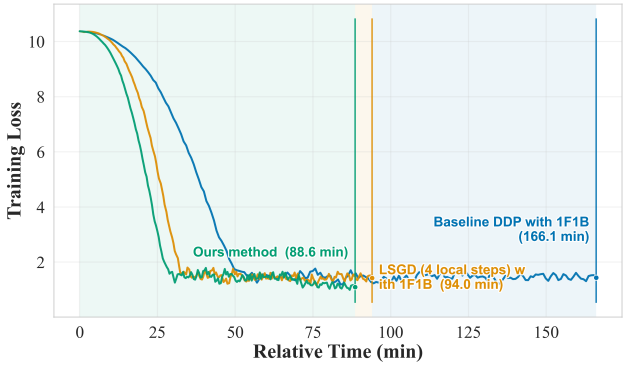

Figure 6. Training loss versus wall-clock time on WikiText-103 (smoothed). POLAR-SGD reaches the low-loss regime substantially earlier than the Baseline DDP, demonstrating improved time-to-target under Cross-DC network constraints.

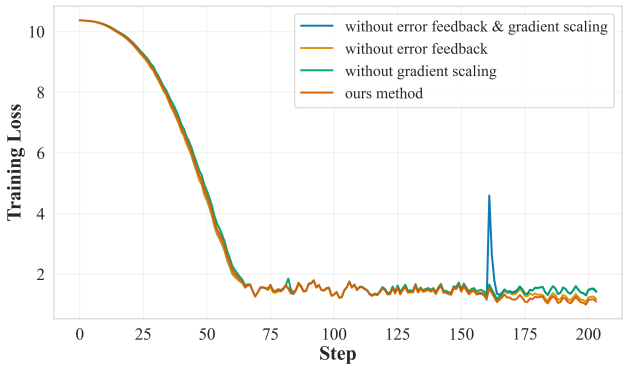

Figure 7. Training loss changes with step under different ablation settings.

degraded convergence in the final loss. For the variant without both mechanisms, obvious spikes appear after training has converged, indicating that the stability of the training is compromised to some extent.

5. Related Work

This section situates POLAR-SGD with respect to the most relevant algorithmic and system approaches for distributed training limited by the WAN, focusing on precise differences in the mechanism and objectives.

5.1. Local Updates / Local-SGD

Local-SGD reduces global communication by executing multiple local SGD steps between periodic aggregations, which lowers aggregate bandwidth but induces "client drift" (Stich, 2019; Stich et al., 2018; Zhou et al., 2022). The sysname function does not primarily sparsify synchronization points; rather, it uses micro-batch granular continuous gradient transmission to avoid model drift.

5.2. Gradient Compression and Error-Feedback

Quantization and sparsification combined with error feedback reduce transmitted bytes while recovering convergence guarantees under certain assumptions (Wang et al., 2025; Modoranu et al., 2024; Bao et al., 2025; Xu & Huang, 2022). Those techniques principally address bias from lossy encoding; POLAR-SGD addresses a distinct class of temporal misalignment errors arising from early transmission of gradients and is compatible with compression + error-feedback applied on its continuous gradient stream.

5.3. Overlap and Pipelining

Overlap techniques and model-parallel pipelining hide communication by interleaving compute and communication or by partitioning layers across devices (Huang et al., 2019; Narayanan et al., 2019; Qi et al., 2024). While such methods can reduce on-step communication tails, cross-DC model pipelining typically requires activation transfers that are prohibitive in WAN environments; unlike opportunistic intra-step overlap, POLAR-SGD proactively moves micro-batch gradients out-of-band into a sustained background flow, avoiding cross-DC activation transfer and converting bursty synchronization into steady, low-jitter traffic.

5.4. Summary and Positioning

POLAR-SGD draws on three related lines of work: micro-batch/overlap techniques, asynchronous and delay-compensation methods, and error-feedback approaches. It departs from these prior lines in three principal ways: (i) it moves synchronization to micro-batch granularity; (ii) it employs predictive correction targeted at the temporal mismatch between previewed gradients and subsequent local updates. The empirical section evaluates POLAR-SGD against representative instances from these families in terms of throughput, convergence speed, and final accuracy under diverse WAN conditions and data heterogeneity.

6. Conclusion

We presented POLAR-SGD, which pipelines gradient communication across micro-batches to overlap synchronization with computation, and stabilizes the resulting asynchrony via a lightweight predictive (error-feedback) correction mechanism.

In a realistic Cross-DC emulation (30 Gb/s bandwidth, 50 ms RTT) on LLaMA-2 7B with 32 A100 GPUs, POLAR-SGD improves training throughput by **1.87**×, while model quality is successfully maintained comparable to the synchronous baseline.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

References

Bao, H., Chen, P., Sun, Y., and Li, Z. Efskip: A new error feedback with linear speedup for compressed federated learning with arbitrary data heterogeneity. In Walsh, T., Shah, J., and Kolter, Z. (eds.), *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*, pp. 15489–15497. AAAI Press, 2025. doi: 10.1609/AAAI.V39I15.33700. URL https://doi.org/10.1609/aaai.v39i15.33700.

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R. B., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., Buch, S., Card, D., Castellon, R., Chatterji, N. S., Chen, A. S., Creel, K., Davis, J. Q., Demszky, D., Donahue, C., Doumbouya, M., Durmus, E., Ermon, S., Etchemendy, J., Ethayarajh, K., Fei-Fei, L., Finn, C., Gale, T., Gillespie, L. E., Goel, K., Goodman, N. D., Grossman, S., Guha, N., Hashimoto, T., Henderson, P., Hewitt, J., Ho, D. E., Hong, J., Hsu, K., Huang, J., Icard, T., Jain, S., Jurafsky, D., Kalluri, P., Karamcheti, S., Keeling, G., Khani, F., Khattab, O., Koh, P. W., Krass, M. S., Krishna, R., Kuditipudi, R., and et al. On the opportunities and risks of foundation models. *CoRR*, abs/2108.07258, 2021. URL https://arxiv.org/abs/2108.07258.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

Chen, T., Kubicek, A., Huang, L., and Hoefler, T. Crosspipe: Towards optimal pipeline schedules for cross-datacenter training. In Altinbüken, D. and Stutsman, R. (eds.), *Proceedings of the 2025 USENIX Annual Technical Conference, USENIX ATC 2025, Boston, MA, USA, July 7-9, 2025*, pp. 1089–1108. USENIX Association, 2025. URL https://www.usenix.org/conference/atc25/presentation/chen-tiancheng.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., Schuh, P., Shi, K., Tsvyashchenko, S., Maynez, J., Rao, A., Barnes, P., Tay, Y., Shazeer, N., Prabhakaran, V., Reif, E., Du, N., Hutchinson, B., Pope, R., Bradbury, J., Austin, J., Isard, M., Gur-Ari, G., Yin, P., Duke, T., Levskaya, A., Ghemawat, S., Dev, S., Michalewski, H., Garcia, X., Misra, V., Robinson, K., Fedus, L., Zhou, D., Ippolito, D., Luan, D., Lim, H., Zoph, B., Spiridonov, A., Sepassi, R., Dohan, D., Agrawal, S., Omernick, M., Dai, A. M., Pillai, T. S., Pellat, M., Lewkowycz, A., Moreira, E., Child, R., Polozov, O., Lee, K., Zhou, Z., Wang, X., Saeta, B., Diaz, M., Firat, O., Catasta, M., Wei, J., Meier-Hellstern, K., Eck, D., Dean, J., Petrov, S., and Fiedel, N. Palm: Scaling language modeling with pathways. *CoRR*, abs/2204.02311, 2022. doi: 10.48550/ARXIV.2204.02311. URL https://doi.org/10.48550/arXiv.2204.02311.

Dai, J., Wang, X., Fang, K., Yang, Z., Ji, Y., and Zhang, J. Geopipe: a geo-distributed LLM training framework with enhanced pipeline parallelism in a lossless rdma-enabled datacenter optical transport network. *CoRR*, abs/2510.12064, 2025. doi: 10.48550/ARXIV.2510.12064. URL https://doi.org/10.48550/arXiv.2510.12064.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.

Deng, H., Song, X., Zhang, M., Yang, Y., and Xu, M. JEEVES: the valet who masters the art of cross-dc training scheduling. In Liu, A. Z., Godfrey, P. B., and Raghavan, B. (eds.), *Proceedings of the 24th ACM Workshop on Hot Topics in Networks, HotNets 2025, UMD Campus, College Park, MD, USA, November 17-18, 2025*, pp. 168–175. ACM, 2025. doi: 10.1145/3772356.3772387. URL https://doi.org/10.1145/3772356.3772387.

Devlin, J., Chang, M., Lee, K., and Toutanova, K. BERT: pre-training of deep bidirectional transformers for language understanding. In Burstein, J., Doran, C., and Solorio, T. (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pp. 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1423. URL https://doi.org/10.18653/v1/n19-1423.

Douillard, A., Donchev, Y., Rush, K., Kale, S., Charles, Z., Garrett, Z., Teston, G., Lacey, D., McIlroy, R., Shen, J., Ramé, A., Szlam, A., Ranzato, M., and Barham, P. Streaming diloco with overlapping communication: Towards a distributed free lunch. *CoRR*, abs/2501.18512,

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

2025. doi: 10.48550/ARXIV.2501.18512. URL https://doi.org/10.48550/arXiv.2501.18512.

Gu, X., Lyu, K., Huang, L., and Arora, S. Why (and when) does local SGD generalize better than sgd? In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL https://openreview.net/forum?id=svCcui6Drl.

Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, D., Chen, M. X., Lee, H., Ngiam, J., Le, Q. V., Wu, Y., and Chen, Z. Gpipe: Efficient training of giant neural networks using pipeline parallelism. In Wallach, H. M., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E. B., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pp. 103–112, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/093f65e080a295f8076b1c5722a46aa2-Abstract.html.

Jaghouar, S., Ong, J. M., and Hagemann, J. Opendiloco: An open-source framework for globally distributed low-communication training. *CoRR*, abs/2407.07852, 2024. doi: 10.48550/ARXIV.2407.07852. URL https://doi.org/10.48550/arXiv.2407.07852.

Karimireddy, S. P., Rebjock, Q., Stich, S. U., and Jaggi, M. Error feedback fixes signsgd and other gradient compression schemes. In Chaudhuri, K. and Salakhutdinov, R. (eds.), *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pp. 3252–3261. PMLR, 2019. URL http://proceedings.mlr.press/v97/karimireddy19a.html.

Karimireddy, S. P., Kale, S., Mohri, M., Reddi, S. J., Stich, S. U., and Suresh, A. T. SCAFFOLD: stochastic controlled averaging for federated learning. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pp. 5132–5143. PMLR, 2020. URL http://proceedings.mlr.press/v119/karimireddy20a.html.

Kloudas, K., Rodrigues, R., Preguiça, N. M., and Mamede, M. PIXIDA: optimizing data parallel jobs in wide-area data analytics. *Proc. VLDB Endow.*, 9(2):72–83, 2015. doi: 10.14778/2850578.2850582. URL http://www.vldb.org/pvldb/vol9/p72-kloudas.pdf.

Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. Gshard: Scaling giant models with conditional computation and automatic sharding. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL https://openreview.net/forum?id=qrwe7XHTmYb.

Li, S., Zhao, Y., Varma, R., Salpekar, O., Noordhuis, P., Li, T., Paszke, A., Smith, J., Vaughan, B., Damania, P., and Chintala, S. Pytorch distributed: Experiences on accelerating data parallel training. *Proc. VLDB Endow.*, 13(12):3005–3018, 2020. doi: 10.14778/3415478.3415530. URL http://www.vldb.org/pvldb/vol13/p3005-li.pdf.

Liang, F., Zhang, Z., Lu, H., Leung, V. C. M., Guo, Y., and Hu, X. Communication-efficient large-scale distributed deep learning: A comprehensive survey. volume abs/2404.06114, 2024. doi: 10.48550/ARXIV.2404.06114. URL https://doi.org/10.48550/arXiv.2404.06114.

Modoranu, I., Kalinov, A., Kurtic, E., Frantar, E., and Alistarh, D. Error feedback can accurately compress preconditioners. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=OJTKlubFk1.

Narayanan, D., Harlap, A., Phanishayee, A., Seshadri, V., Devanur, N. R., Ganger, G. R., Gibbons, P. B., and Zaharia, M. Pipedream: generalized pipeline parallelism for DNN training. In Brecht, T. and Williamson, C. (eds.), *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP 2019, Huntsville, ON, Canada, October 27-30, 2019*, pp. 1–15. ACM, 2019. doi: 10.1145/3341301.3359646. URL https://doi.org/10.1145/3341301.3359646.

Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., Phanishayee, A., and Zaharia, M. Efficient large-scale language model training on GPU clusters using megatron-lm. pp. 58, 2021. doi: 10.1145/3458817.3476209. URL https://doi.org/10.1145/3458817.3476209.

Qi, P., Wan, X., Huang, G., and Lin, M. Zero bubble (almost) pipeline parallelism. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=tuzTN0eIO5.

Rajbhandari, S., Rasley, J., Ruwase, O., and He, Y. Zero: memory optimizations toward training trillion parameter models. In Cuicchi, C., Qualters, I., and Kramer, W. T. (eds.), *Proceedings of the International Conference for*

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020, pp. 20. IEEE/ACM, 2020. doi: 10.1109/SC41405.2020.00024. URL https://doi.org/10.1109/SC41405.2020.00024.

Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-lm: Training multi-billion parameter language models using model parallelism. *CoRR*, abs/1909.08053, 2019. URL http://arxiv.org/abs/1909.08053.

Stich, S. U. Local SGD converges fast and communicates little. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. URL https://openreview.net/forum?id=S1g2JnRcFX.

Stich, S. U., Cordonnier, J., and Jaggi, M. Sparsified SGD with memory. In Bengio, S., Wallach, H. M., Larochelle, H., Grauman, K., Cesa-Bianchi, N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pp. 4452–4463, 2018. URL https://proceedings.neurips.cc/paper/2018/hash/b440509a0106086a67bc2ea9df0a1dab-Abstract.html.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971, 2023. doi: 10.48550/ARXIV.2302.13971. URL https://doi.org/10.48550/arXiv.2302.13971.

Wang, Y., Wu, R., Li, X., and Kutscher, D. Pactrain: Pruning and adaptive sparse gradient compression for efficient collective communication in distributed deep learning. In *62nd ACM/IEEE Design Automation Conference, DAC 2025, San Francisco, CA, USA, June 22-25, 2025*, pp. 1–7. IEEE, 2025. doi: 10.1109/DAC63849.2025.11133419. URL https://doi.org/10.1109/DAC63849.2025.11133419.

Xu, A. and Huang, H. Detached error feedback for distributed SGD with random sparsification. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvári, C., Niu, G., and Sabato, S. (eds.), *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pp. 24550–24575. PMLR, 2022. URL https://proceedings.mlr.press/v162/xu22c.html.

Xu, A., Huo, Z., and Huang, H. Step-ahead error feedback for distributed training with compressed gradient. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*, pp. 10478–10486. AAAI Press, 2021. doi: 10.1609/AAAI.V35I12.17254. URL https://doi.org/10.1609/aaai.v35i12.17254.

Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., Desmaison, A., Balioglu, C., Damania, P., Nguyen, B., Chauhan, G., Hao, Y., Mathews, A., and Li, S. Pytorch FSDP: experiences on scaling fully sharded data parallel. *Proc. VLDB Endow.*, 16(12):3848–3860, 2023. doi: 10.14778/3611540.3611569. URL https://www.vldb.org/pvldb/vol16/p3848-huang.pdf.

Zhou, Y., Ye, Q., and Lv, J. Communication-efficient federated learning with compensated overlap-fedavg. *IEEE Trans. Parallel Distributed Syst.*, 33(1):192–205, 2022. doi: 10.1109/TPDS.2021.3090331. URL https://doi.org/10.1109/TPDS.2021.3090331.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

A. Additional Details

A.1. Code base

Our code is published at: https://anonymous.4open.science/r/polar-sgd-7583.

A.2. Rethinking Parallelism Choices in Cross-DC Training

We can make the above distinction precise by comparing how cross-datacenter communication scales with the global batch size B.

Consider D datacenters, each processing local minibatch b such that $B = D \cdot b$, and let α be bytes per parameter communicated. Let A denote the bytes of activation tensors that must cross a pipeline boundary *per sample* when a pipeline cut spans datacenters (this includes the forward activations and, correspondingly, backward activation gradients on the return path).

Cross-DC DP (gradient synchronization). In data parallel training, each datacenter performs forward/backward locally and participates in a global gradient aggregation once per iteration. The amount of data communicated across datacenters for the aggregation is proportional to the model size, not the batch size:

$$V_{\mathrm{DP}} \approx \alpha P$$

Thus, with fixed model size, the WAN communication volume for DP is *constant* as B increases.

Cross-DC PP (activation transfer). When pipeline stages are placed in different datacenters, each microbatch must transmit activations across the WAN at each cross-DC boundary. Aggregated over the iteration, the WAN traffic scales with the number of samples processed:

$$V_{\mathrm{PP}} \approx \Theta(B \cdot A),$$

meaning cross-DC PP incurs WAN communication that grows *linearly* with the global batch size.

Implication on iteration time and throughput. Let $\mathrm{BW}_{\mathrm{WAN}}$ be the effective WAN bandwidth and $\mathrm{Lat}_{\mathrm{WAN}}$ the WAN latency. A coarse lower bound on communication time is:

$$T_{\mathrm{comm}} \approx \mathrm{Lat}_{\mathrm{WAN}} + \frac{V}{\mathrm{BW}_{\mathrm{WAN}}}.$$

Therefore,

$$T_{\mathrm{comm}}^{\mathrm{DP}} \approx \mathrm{Lat}_{\mathrm{WAN}} + \frac{\alpha P}{\mathrm{BW}_{\mathrm{WAN}}},$$
$$T_{\mathrm{comm}}^{\mathrm{PP}} \approx \mathrm{Lat}_{\mathrm{WAN}} + \frac{c\,B A}{\mathrm{BW}_{\mathrm{WAN}}},$$

where c captures constants such as the number of cross-DC cuts and forward/backward directions.

As B increases, $T_{\mathrm{comm}}^{\mathrm{PP}}$ increases proportionally, and more importantly, PP communication lies on the critical path due to stage ordering constraints, so WAN variability directly stalls compute. In contrast, $T_{\mathrm{comm}}^{\mathrm{DP}}$ does not grow with B; the compute per iteration typically increases with B, so the fixed WAN synchronization cost is amortized over more computation and samples. Consequently, end-to-end throughput of DP-centric Cross-DC training becomes increasingly favorable at large batch sizes, while PP-centric Cross-DC training suffers growing WAN overhead and sensitivity to network dynamics.

A.3. Proofs

We provide a convergence analysis showing that POLAR-SGD retains the same *asymptotic* convergence behavior as synchronous SGD, up to an additional term controlled by the error-feedback residual.

A.3.1. Assumptions

We make the following standard assumptions in stochastic optimization:

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Assumption A.1 (L-Smoothness)**.** The loss function $f(w)$ is L-smooth, i.e., for all w, w':

$$\|\nabla f(w) - \nabla f(w')\| \leq L\|w - w'\|. \tag{7}$$

Assumption A.2 (Unbiased Gradient)**.** For each worker r and micro-batch m, the stochastic gradient is unbiased:

$$\mathbb{E}\left[g_{r,t,m} \mid w_t\right] = \nabla f(w_t). \tag{8}$$

Assumption A.3 (Bounded Variance)**.** The variance of stochastic gradients is bounded:

$$\mathbb{E}\left[\|g_{r,t,m} - \nabla f(w_t)\|^2 \mid w_t\right] \leq \sigma^2. \tag{9}$$

A.3.2. Main Convergence Theorem

Theorem A.4 (Convergence of POLAR-SGD (mean All-Reduce with error feedback))**.** *Under Assumptions A.1–A.3, with learning rate $\eta \leq \frac{1}{2L}$, after T global iterations, POLAR-SGD satisfies*

$$\begin{aligned}\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\left[\|\nabla f(w_t)\|^2\right] \leq \frac{2\big(f(w_0) - f^*\big)}{\eta T} + \frac{L\eta\sigma^2}{N} \\ + \frac{L\eta}{T}\sum_{t=0}^{T-1}\mathbb{E}\left[\|\bar{e}_t\|^2\right],\end{aligned} \tag{10}$$

where $f^* = \inf_w f(w)$ and $\bar{e}_t = \frac{1}{N}\sum_{r=1}^{N} e_{r,t}$ is the averaged error buffer.

A.4. Detailed Proof

A.4.1. Key Definitions (Aligned with Algorithm 2)

For a global iteration with M micro-batches, we define:

- i: Index of the micro-batch where communication is triggered ($1 \leq i \leq M$)
- $g_a^{(i)} = \sum_{k=1}^{i} g_k$: Local accumulated gradient from micro-batch 1 to i
- $g_{\text{pred}}^{(i)} = \frac{M}{i} \cdot g_a^{(i)}$: Predicted full-batch gradient (scaled by $\frac{M}{i}$ to match the norm of the full M micro-batches)
- $e^{(i)} = g_a^{(M)} - g_{\text{pred}}^{(i)}$: Error buffer (compensation term for the unsynchronized micro-batches $i+1$ to M)
- $g_{\text{sync}}^{(i)} = \frac{1}{N}\sum_{j=1}^{N}\left(g_{\text{pred},j}^{(i)} + e_j^{(i-1)}\right)$: Global synchronized gradient (incorporating error feedback from the previous step)

A.4.2. Assumptions (Revised for Asynchronous Scenarios)

We retain the three standard assumptions and add one asynchronous bias constraint to match POLAR-SGD's pipeline communication:

1. L**-Smoothness**: $\|\nabla f(w_1) - \nabla f(w_2)\| \leq L\|w_1 - w_2\|$ for all w_1, w_2
2. **Unbiased Gradient**: $\mathbb{E}[g_k \mid w_k] = \nabla f(w_k)$, where w_k is the parameter used for micro-batch k
3. **Bounded Variance**: $\mathbb{E}\left[\|g_k - \nabla f(w_k)\|^2 \mid w_k\right] \leq \sigma^2$
4. **Bounded Asynchronous Bias**: Due to pipeline communication, w_k may differ from the globally synchronized parameter w_{sync}. We assume $\|w_k - w_{\text{sync}}\| \leq \Delta$, where Δ is a constant determined by communication-computation overlap ratio.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

A.4.3. Proof Steps

Step 1: Error Buffer Dynamics and Boundedness The error buffer $e^{(i)}$ is defined as the deviation between the full local accumulated gradient $g_a^{(M)}$ and the predicted gradient $g_{\text{pred}}^{(i)}$ sent for synchronization:

$$e^{(i)} = \sum_{k=1}^{M} g_k - \frac{M}{i}\sum_{k=1}^{i} g_k = \sum_{k=i+1}^{M} g_k - \frac{M-i}{i}\sum_{k=1}^{i} g_k \tag{11}$$

From the bounded variance assumption, each g_k satisfies $\|g_k\| \leq G$ (a constant gradient norm bound for LLMs). Thus:

$$\|e^{(i)}\| \leq \sum_{k=i+1}^{M} \|g_k\| + \frac{M-i}{i}\sum_{k=1}^{i} \|g_k\| \leq M \cdot G \tag{12}$$

This proves the boundedness of $e^{(i)}$: the error buffer has an upper bound $M \cdot G$ and lower bound $-M \cdot G$, which is independent of the number of global steps—it only depends on the number of micro-batches M and the single micro-batch gradient norm G.

For multiple workers, the sum of errors across workers $\sum_{j=1}^{N} e_j^{(i)}$ is not necessarily zero—it depends on the randomness of local gradients across workers. However, due to the unbiasedness of local gradients, we can derive the expectation bound of the sum:

$$\mathbb{E}\left[\left\|\sum_{j=1}^{N} e_j^{(i)}\right\|^2\right] \leq N \cdot M^2 \cdot G^2 \tag{13}$$

Step 2: Effective Gradient and Asynchronous Bias Compensation The global synchronized gradient incorporating error feedback is:

$$g_{\text{sync}}^{(i)} = \frac{1}{N}\sum_{j=1}^{N}\left(\frac{M}{i} g_{a,j}^{(i)} + e_j^{(i-1)}\right) \tag{14}$$

Substitute $e_j^{(i-1)} = g_{a,j}^{(M)} - \frac{M}{i-1} g_{a,j}^{(i-1)}$ (from the previous step's error definition):

$$g_{\text{sync}}^{(i)} = \frac{1}{N}\sum_{j=1}^{N}\left(\frac{M}{i} g_{a,j}^{(i)} + g_{a,j}^{(M)} - \frac{M}{i-1} g_{a,j}^{(i-1)}\right) \tag{15}$$

When $i = M$ (synchronization after all micro-batches), the predicted gradient equals the full local gradient, so $e_j^{(M)} = 0$. For general i, the effective gradient $g_{\text{sync}}^{(i)}$ can be rewritten as:

$$g_{\text{sync}}^{(i)} = \frac{1}{N}\sum_{j=1}^{N} g_{a,j}^{(M)} + \delta^{(i)} \tag{16}$$

where $\delta^{(i)}$ is the asynchronous bias term caused by early synchronization. From the boundedness of $e^{(i)}$, we have $\|\delta^{(i)}\| \leq C \cdot \Delta$ (a constant C related to M).

Step 3: Iteration Inequality Under Smoothness The parameter update rule for POLAR-SGD is:

$$w^{(m+1)} = w^{(m)} - \eta \cdot g_{\text{sync}}^{(i,m)} \tag{17}$$

where $g_{\text{sync}}^{(i,m)}$ is the synchronized gradient for micro-batch m in the global iteration.

By L-smoothness:

$$f(w^{(m+1)}) \leq f(w^{(m)}) + \langle \nabla f(w^{(m)}), w^{(m+1)} - w^{(m)} \rangle + \frac{L}{2}\|w^{(m+1)} - w^{(m)}\|^2 \tag{18}$$

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Substitute the update rule and expand:

$$f(w^{(m+1)}) \leq f(w^{(m)}) - \eta\langle \nabla f(w^{(m)}), g_{\text{sync}}^{(i,m)}\rangle + \frac{L\eta^2}{2}\|g_{\text{sync}}^{(i,m)}\|^2 \tag{19}$$

Step 4: Expectation and Convergence Bound Take expectation over gradient randomness and asynchronous bias:

$$\mathbb{E}[f(w^{(m+1)})] \leq \mathbb{E}[f(w^{(m)})] - \eta\mathbb{E}\left[\langle \nabla f(w^{(m)}), g_{\text{sync}}^{(i,m)}\rangle\right] + \frac{L\eta^2}{2}\mathbb{E}\left[\|g_{\text{sync}}^{(i,m)}\|^2\right] \tag{20}$$

From the unbiased gradient assumption and bounded asynchronous bias:

$$\mathbb{E}\left[\langle \nabla f(w^{(m)}), g_{\text{sync}}^{(i,m)}\rangle\right] = \|\nabla f(w^{(m)})\|^2 + \mathbb{E}\left[\langle \nabla f(w^{(m)}), \delta^{(i,m)}\rangle\right] \geq \|\nabla f(w^{(m)})\|^2 - D\cdot\Delta \tag{21}$$

where D is the Lipschitz constant of ∇f.

For the second moment term, using bounded variance and error boundedness:

$$\mathbb{E}\left[\|g_{\text{sync}}^{(i,m)}\|^2\right] \leq \|\nabla f(w^{(m)})\|^2 + \frac{\sigma^2}{N} + C^2\cdot\Delta^2 \tag{22}$$

Substitute back into the iteration inequality and rearrange:

$$\mathbb{E}[f(w^{(m+1)})] \leq \mathbb{E}[f(w^{(m)})] - \eta\left(1 - \frac{L\eta}{2}\right)\|\nabla f(w^{(m)})\|^2 + \eta D\Delta + \frac{L\eta^2}{2}\left(\frac{\sigma^2}{N} + C^2\Delta^2\right) \tag{23}$$

Set the learning rate condition $\eta \leq \frac{1}{2L}$ to ensure the coefficient of $\|\nabla f(w^{(m)})\|^2$ is positive:

$$1 - \frac{L\eta}{2} \geq \frac{3}{4} \tag{24}$$

Step 5: Summing Over Steps and Final Convergence Bound Sum over M micro-batches per global iteration and T global iterations:

$$\sum_{t=0}^{T-1}\sum_{m=0}^{M-1}\mathbb{E}\left[\|\nabla f(w_t^{(m)})\|^2\right] \leq \frac{4}{\eta}\left(f(w_0) - f^* + TM\left(\eta D\Delta + \frac{L\eta^2}{2}\left(\frac{\sigma^2}{N} + C^2\Delta^2\right)\right)\right) \tag{25}$$

Divide both sides by TM to get the average gradient norm bound:

$$\frac{1}{TM}\sum_{t=0}^{T-1}\sum_{m=0}^{M-1}\mathbb{E}\left[\|\nabla f(w_t^{(m)})\|^2\right] \leq \frac{4(f(w_0) - f^*)}{\eta TM} + 4D\Delta + \frac{2L\eta}{N}\sigma^2 + 2L\eta C^2\Delta^2 \tag{26}$$

Set the learning rate $\eta = \frac{K}{\sqrt{TM}}$ (to balance convergence rate and bias) where $K \leq \frac{\sqrt{TM}}{2L}$. The convergence rate is $O(1/\sqrt{TM})$, which matches standard synchronous SGD. The additional terms ($4D\Delta$, $2L\eta C^2\Delta^2$) are constants determined by asynchronous bias, which vanish as $\Delta \to 0$ (full communication-computation overlap).

This completes the proof.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.